SPARSE ATTENTION ADAPTATION FOR LONG REASONING

Yizhao Gao[1,2*†] **Shuming Guo**[3*†] **Shijie Cao**[2‡] **Yuqing Xia**[2] **Yu Cheng**[4] **Lei Wang**[4]
Lingxiao Ma[2] **Yutao Sun**[5] **Tianzhu Ye**[2] **Li Dong**[2] **Hayden Kwok-Hay So**[1]
Yu Hua[3] **Ting Cao**[5] **Fan Yang**[2] **Mao Yang**[2]

[1]The University of Hong Kong
[2]Microsoft Research
[3]Huazhong University of Science and Technology
[4]Peking University
[5]Tsinghua University

ABSTRACT

We introduce SeerAttention-R, a sparse attention framework specifically tailored for the long decoding of reasoning models. Extended from SeerAttention, SeerAttention-R retains the design of learning attention sparsity through a self-distilled gating mechanism, while removing query pooling to accommodate auto-regressive decoding. With a **lightweight** plug-in gating, SeerAttention-R is **flexible** and can be easily integrated into existing pretrained model without modifying the original parameters. We demonstrate that SeerAttention-R, trained on just 0.4B tokens, maintains near-lossless reasoning accuracy with 4K token budget in AIME benchmark under large sparse attention block sizes (64/128). Using TileLang, we develop a highly optimized sparse decoding kernel that achieves near-theoretical speedups of up to 9x over FlashAttention-3 on H100 GPU at 90% sparsity.

1 INTRODUCTION

Recent reasoning-focused models such as OpenAI o1 Jaech et al. (2024), DeepSeek-R1 Guo et al. (2025), and Qwen3 Yang et al. (2025a) demonstrate that models' capabilities improve significantly through test-time scaling. By generating longer sequences during inference, these models are able to think and reason more effectively before producing an answer. Empirically, longer generations correlate with stronger reasoning performance. For instance, Qwen3-14B Yang et al. (2025a) outperforms DeepSeek-R1-Distill-Qwen-14B Guo et al. (2025) while producing longer responses on average. Similarly, harder benchmarks such as AIME24 require more tokens per generation than easier ones like MATH-500 Hendrycks et al. (2020).

However, deeper reasoning introduces increasing efficiency challenges. Due to the auto-regressive nature of decoding, later tokens must attend to a longer context, increasing compute and memory demands for the KV cache. As a result, the per-token generation cost grows linearly, while the overall generation cost increases quadratically.

Sparse attention offers a promising approach to addressing the long-sequence efficiency challenges. While it has been studied in general language modeling, its application to reasoning models, which require prolonged decoding, remains underexplored. Our experiment using oracle sparsity (Section 4.2) shows that attention in reasoning models is also inherently sparse, activating only a subset of important tokens is sufficient to maintain the model's reasoning capability. The key challenge lies in effectively identifying and leveraging this intrinsic sparsity.

[*]Equal Contribution.
[†]Work done during internship at Microsoft Research Asia.
[‡]Corresponding author.

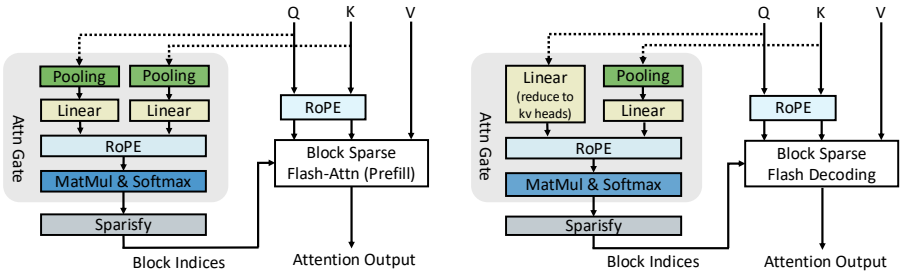
In this work, we extend SeerAttention Gao et al. (2024) to SeerAttention-R, a sparse attention framework aimed to improve the long decoding efficiency of reasoning models. SeerAttention was originally designed to improve prefill efficiency by selectively activating important attention blocks through a lightweight, self-distilled attention gating mechanism at post-training time. SeerAttention-R retains the core design of self-distilled attention sparsity and introduces modifications to support efficient decoding. Specifically, it removes sequence-level pooling of query to accommodate auto-regressive decoding and adopts a shared sparsity design aligned with Grouped Query Attention (GQA) to enhance hardware efficiency. SeerAttention-R can be integrated into any standard transformer-based pretrained model by adding the learnable gate to the attention layer, without fine-tuning original model parameters.

We apply SeerAttention-R to multiple reasoning-focused open-source models, including Qwen3-4B, 8B, 14B Yang et al. (2025a) and DeepSeek-R1-Distill-Qwen-14B Guo et al. (2025), and evaluate them on several reasoning benchmarks: AIME24, AIME25, MATH-500 Hendrycks et al. (2020), and GPQA-Diamond Rein et al. (2024). Since SeerAttention-R only requires training the gating module, the distillation is lightweight with just 0.4B tokens from OpenR1-MATH-220K Face (2025) being sufficient. Across all models and tasks, SeerAttention-R consistently outperforms the Quest Tang et al. (2024) baseline and maintains near-lossless accuracy under a 4k token budget. Notably, the accuracy gap further diminishes as model size increases. More importantly, this learnable approach enables more **coarse-grained sparse attention** (e.g., a block size of 64 or 128), which further reduces the overhead from sparse attention scheme and improve hardware efficiency.

We implement the block sparse flash decoding kernel using both TileLang til and Triton Tillet et al. (2019), and benchmark it on an H100 GPU with FlashAttention-3 (FA3) Shah et al. (2024) as the baseline. Across a range of combination of sequence lengths, batch sizes, and sparsity levels, our TileLang-based kernel consistently outperforms both Triton and FA3. The gains are especially pronounced at large sequence lengths and batch sizes. For example, at batch size 16 and sequence length $\geq$ 32k, our TileLang kernel achieves near-theoretical speedups of up to $8.6\times$ at 90% sparsity over the FA3 baseline, and delivers a $1.7\times$ speedup compared to the Triton counterpart.

2 SeerAttention-R

2.1 A Recap of SeerAttention



(a) SeerAttention **(b) SeerAttention-R**

Figure 1: SeerAttention (Sparse Prefill) and SeerAttention-R (Sparse Decode). In SeerAttention-R, no sequence dimension compression/pooling operation is applied in Query (Q). Given that modern architectures predominantly use GQA, a linear layer projects the Q from its original number of heads down to the number of KV heads, enabling shared sparsity selection in a GQA group.

SeerAttention Gao et al. (2024) introduces self-distilled *Attention Gate* (AttnGate) that dynamically activates sparse blocks in attention computation for efficient long-context **prefilling**. Figure 1a shows the AttnGate architecture of SeerAttention, where $\mathbf{Q}$, $\mathbf{K}$ tensors are both compressed (pooled) in the sequence dimension per block number of tokens. The compressed $\mathbf{Q}$, $\mathbf{K}$ tensors are then passed through two newly added linear layers, which serve as learnable parameters in the AttnGate. With the following positional embedding, matrix-multiplication and softmax operation similar to standard attention, the AttnGate then generates the 2D block-level attention score estimation. Based on the output, we can selectively activate blocks with higher scores while skipping the rest.

In the distillation process, the AttnGate are trained to mimic the 2D block sparse distribution using the ground truth generated by the original pretrained model. This self-distillation training is efficient as the original model weights are frozen. In this way, it brings accurate sparse attention to pretrained full-attention models without costly fine-tuning or pre-training. Powered by customized block-sparse flash attention kernels, SeerAttention achieves supreme accuracy-efficiency tradeoff in downstream long-context benchmarks.

2.2 SEERATTENTION-R: ATTNGATE FOR SPARSE DECODING

This work introduces **SeerAttention-R**, an extension of SeerAttention tailored for the long-decoding phase of reasoning models. The foremost difference of AttnGate design in SeerAttention-R is that it does not apply compression/pooling in the sequence dimension of $\mathbf{Q}$ to accommodate the token-by-token auto-regressive decoding process (shown in Figure 1b).

$$\mathbf{Q_{gate}} = \mathrm{RoPE}\Big(\mathbf{W^{q}_{gate}}\ \mathrm{reshape}(\mathbf{Q_{nope}}, [..., g \cdot d])\Big), \tag{1a}$$

$$\mathbf{K_{gate}} = \mathrm{RoPE}\Big(\mathbf{W^{k}_{gate}}\ \mathrm{concat}[\mathrm{P}_{\max}(\mathbf{K_{nope}}), \mathrm{P}_{\min}(\mathbf{K_{nope}}), \mathrm{P}_{\mathrm{avg}}(\mathbf{K_{nope}})]\Big), \tag{1b}$$

$$\mathbf{S} = \mathrm{softmax}(\mathbf{Q_{gate}}\,\mathbf{K_{gate}}^{\top}/\sqrt{d_{gate}}). \tag{1c}$$

where, $\mathrm{P}_{\mathrm{max}}$, $\mathrm{P}_{\mathrm{min}}$, and $\mathrm{P}_{\mathrm{avg}}$ stand for Max, Min and Average Pooling in sequence dimension, and g is the group size of GQA setting. d and d_{gate} are the hidden dimension of the original model and AttnGate for each head, respectively. $\mathbf{S}$ is the output score of each block from AttnGate. The detailed design are discussed as follows.

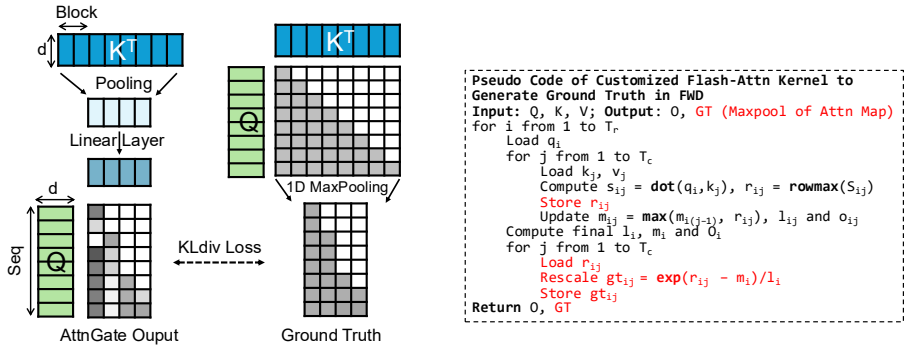
Aggregation of Query Heads for Shared Sparsity in GQA Group Query Attention (GQA) Ainslie et al. (2023) is widely used in LLMs to reduce KV cache size. In GQA, the query heads are organized into groups, and each group shares a key-value head. Recent sparse attention works SAAP Mazaré et al. (2025b) and NSA Yuan et al. (2025) show that using identical attention sparsity choices for all queries in a group can improve the efficiency while achieving similar or better performance. In SeerAttention-R, we follow this practice and use a linear layer in the $\mathbf{Q}$ branch of AttnGate to reduce each subgroup of queries to one single head. For example, with 32 query heads and 8 key-value heads (group size $g = 4$), there will be 8 sets of linear weights in shape $[d_{gate}, 4 \times d]$ applying on each group of queries heads, resulting only 8 heads of $\mathbf{Q_{gate}}$. Since we keep the number of heads untouched in $\mathbf{K}$ branch of AttnGate, the final output of AttnGate will be key-value heads, achieving a shared decision of sparsity in a group.

Pooling-based Compression of Key We follow the practice of SeerAttention that uses pooling operations to compress the sequence dimension of $\mathbf{K}$. The kernel and stride size of pooling are both equal to block size, which can also be understood as non-overlapping chunk-level pooling. To mitigate the potential information loss associated with pooling operations, we employ a composition of Max, Min, and Average pooling operations. The outputs from these pooling operations are concatenated prior to being fed into the subsequent linear layer, similar to SeerAttention. The intuition behind this approach is that Max and Min Pooling can effectively capture outlier values, while Average Pooling helps to keep the overall distribution intact.

Positional Embedding in AttnGate In line with SeerAttention, the decode AttnGate utilizes the pre-rope $\mathbf{Q}$ and $\mathbf{K}$ tensors as inputs and reapplies RoPE Su et al. (2024) within AttnGate. Given that the branch is compressed along the sequence dimension, the position index is assigned to the initial token of each block. In our experiment, we found that the use of positional embedding in AttnGate can consistently achieve better accuracy compared to the design without positional embedding.

2.3 DISTILLATION/TRAINING

Previous SeerAttention introduces AttnGate distillation method using the ground truth generate by LLM itself in the prefilling phase. The training process is efficient as only the AttnGate are trained. In SeerAttention-R, we extend this method to the decoding scenario by slightly changing the form of the ground truth. Figure 2 shows the overall diagram of the training process.



(a) (b)

Figure 2: Training Diagram and Training Kernel of SeerAttention-R. (a) Self-distillation training of AttnGate in SeerAttention-R. It uses 1D maxpooled attention scores from original model as ground truth to train AttnGate. Query head reduction is not plotted in the diagram for simplicity. (2) Pseudo code of attention forward kernel for training that directly generates ground truth and attention output.

Ground Truth To train AttnGate for the auto-regressive decoding process, we need to adapt the ground truth generation method. Instead of performing 2D maxpooling of attention map in the prefill case, we only do column-wise 1D maxpooling shown in Figure 2a. This corresponds to the decoding AttnGate that does not compress in sequence dimension. Moreover, to accommodate the shared sparsity in GQA, the column-pooled attention map is further maxpooled within each query heads subgroup, resulting in a ground truth with key-value heads. Finally, the ground truth is normalized to sum to 1. We then use the Kullback-Leibler divergence loss Joyce (2011) to train AttnGate in the distillation process.

Efficiently Obtaining Ground Truth during Training Explicitly calculating the full attention map softmax($\mathbf{QK^T}/\sqrt{\mathbf{d}}$) and then perform the block-level pooling can cost huge GPU memory due to the quadratic complexity. In SeerAttention-R, we also provide an efficient modification of FlashAttention-2 Dao (2023) kernel that directly generates the ground truth along with the attention output. This kernel largely reuses the intermediate results (e.g. block-level rowmax) in Flash-Attention and thus increases the efficiency of the distillation process. The pseudo code is shown in Figure 2b.

3 INFERENCE OF SEERATTENTION-R

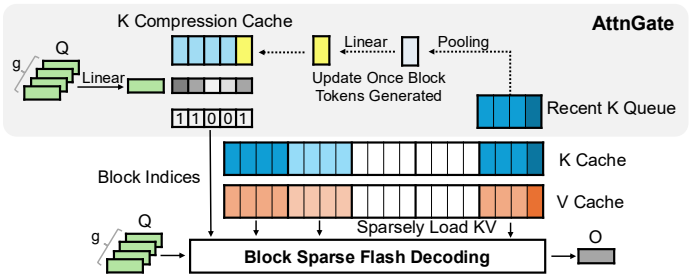


Figure 3: Inference Diagram of SeerAttention-R. During inference, a K Compression Cache is used to cache the compressed key representation in AttnGate to speedup sparse block prediction. This K Compression Cache only updates once per block number of tokens is generated (block=4 in the plots for illustration). As a result, the last block of sequence is always selected to compensate when the compression cache has not been updated yet. g is the group size of GQA.

3.1 SPARSIFY METHODS: TOKEN BUDGET VS THRESHOLD

During training, the AttnGate output $\mathbf{S}$ is distilled to mimic the distribution of the block-wise attention maps from the original model in real-valued (floating-point) form. During inference, important key-value blocks can be selectively activated based on the predictions of AttnGate. In SeerAttention-R, we apply two sparsity methods to convert the soft AttnGate outputs into binary block masks (or block indices). The first method is the *token budget* approach, which is widely adopted in sparse attention methods. Given a fixed token budget, it is first translated into a block budget by dividing the token budget by the block size. The AttnGate outputs are then sorted using a Top-k kernel, where k corresponds to the block budget. While this method introduces an additional Top-k operation, it eliminates the need for a softmax operation in AttnGate. The second method is the *threshold* approach, which simply selects blocks whose scores exceed a given threshold. The threshold method is more self-adaptive as different heads may automatically infer different sparsity ratios. While these two methods involve different trade-offs between efficiency and accuracy, the token budget approach is better suited for direct comparisons with other methods.

3.2 K COMPRESSION CACHE

Similar to KV cache, in SeerAttention-R, we use a *K Compression Cache* to store the compressed representation of K (after pooling plus linear) to speedup AttnGate prediction. Thus, AttnGate does not need to recompute K branch for past seen tokens. The update of K Compression Cache consists of two phases. First, when the sequence length is not a multiple of block size b, the new entry of K Compression Cache may not be accurate. During this time, the last block is always activated to eliminate unnecessary accuracy loss. Second, as long as b number of new tokens are generated, the most recent b tokens will pass through the pooling and linear layer and update the K Compression Cache. In this way, the overhead of AttnGate can be minimized.

In practice, SeerAttention-R utilizes a relatively large block size b, such as 64, which significantly reduces the overhead of the K Compression Cache. Specifically when $b = 64$, the additional memory required for the K Compression Cache amounts to only 1/128 (<1%) of the original KV cache size. This minimal overhead makes it highly efficient. Moreover, it introduces the possibility of offloading the larger KV cache to CPU or other storage. During inference, only the activated blocks need to be retrieved and transferred back to GPU memory on demand. Alternatively, sparse attention computations can even be performed on heterogeneous resources, such as the CPU, further optimizing memory usage and enabling efficient handling of long-context decoding tasks.

3.3 BLOCK SPARSE FLASH DECODING KERNEL

To accelerate decoding under block-sparse attention, we design a specialized kernel that extends the FlashAttention decoding pattern to support dynamic block sparsity in the key/value memory. Our kernel adopts the grid scheduling strategy of flash decoding for GQA, using a three-dimensional launch space over (*batch, heads_kv, num_split*). This design supports concurrent computation across multiple query groups and key/value shards, maximizing block-level parallelism.

Our block sparse version of the decoding kernel takes the activated block indices from AttnGate (shape [*batch, heads_kv, max_selected_blocks*]), which encodes the selected key/value blocks for each group of query heads. During execution, the kernel only traverses the selected indices and thus skips invalid entries, avoiding unnecessary computation and memory access. To improve load/compute balancing across Streaming Multiprocessors (SMs), we partition the key/value blocks along the *num_split* dimension using *max_selected_blocks* rather than the total number of blocks. This strategy ensures a more uniform work distribution in the presence of sparsity-induced irregularity.

On H100 GPUs, our kernel leverages the *wgmma* instructions for better Tensor Core usage by padding the number of query head groups to 64. We implement the kernel using TileLang til, which automatically applies computation optimizations like tiling Zhu et al. (2022), warp specialization and pipelining Cheng et al. (2025), and memory layout optimizations such as tensorization, rasterization and swizzling Wang et al. (2024) based on the target architecture. Additionally, we provide a Triton-based implementation with the same scheduling strategy, allowing for comparative evaluation.

4 EXPERIMENTS

4.1 EXPERIMENTS SETUP

Benchmarks, Models, and Baselines We evaluate SeerAttention-R on three math reasoning benchmarks: the American Invitational Mathematics Examination: AIME24, AIME25, and MATH-500 Hendrycks et al. (2020), as well as GPQA-Diamond Rein et al. (2024). For model evaluation, we select four open-source pre-trained language models with strong reasoning capabilities: Qwen3-4B, 8B, 14B Yang et al. (2025a), and DeepSeek-R1-Distill-Qwen-14B Guo et al. (2025). All models are based on the standard Transformer architecture with Grouped Query Attention (GQA). We compare SeerAttention-R against standard full attention and Quest Tang et al. (2024). Quest is a training-free sparse attention algorithm applied during decoding, employing a query-aware key-value (KV) cache selection strategy. Specifically, Quest estimates the upper bound of attention scores within each KV block (or "page") to select the most relevant blocks. By default, Quest uses a block size of 16, and keeps the first two layers fully dense to minimize error. In Section 4.3, we set the block size to 64 for both Quest and SeerAttention-R, and apply sparse attention to all layers to enable a direct comparison. We also conduct ablation studies to analyze the impact of varying block sizes (Appendix A.1) and incorporating hybrid dense layers (Appendix A.2). Note that SeerAttention-R enables shared sparsity selection within each GQA group, whereas Quest does not. Across all experiments, we set the max output length to 32,768 tokens. While Qwen3 series of models extends this length to 38,912 for AIME24 and AIME25 in their official report, we fix this output length to ensure consistency and fair comparison across all settings. For SeerAttention-R and the full attention baseline, we report average pass@1 accuracy over 64 samples for AIME24 and AIME25, 8 samples for MATH-500, and 16 samples for GPQA-Diamond.

Training Setup for SeerAttention-R To distill AttnGate, we use the OpenR1-MATH-220k Face (2025) dataset for training. Importantly, only the AttnGate is trained, and the original model weights remain unchanged. Inputs are packed into sequences of up to 32k tokens with our variable-length Flash-Attention training kernel that also generates ground truth (Section 2.3). Training is performed with a global batch size of 16 for 800 steps on AMD MI300x GPUs, utilizing DeepSpeed ZeRO-2 optimization. We use AdamW optimizer and a learning rate of 1e-3 with cosine decay schedule; a higher learning rate is feasible here because only the AttnGate parameters are trained.

4.2 ORACLE SPARSE ACCURACY: HOW SPARSE IS ATTENTION IN REASONING MODELS?

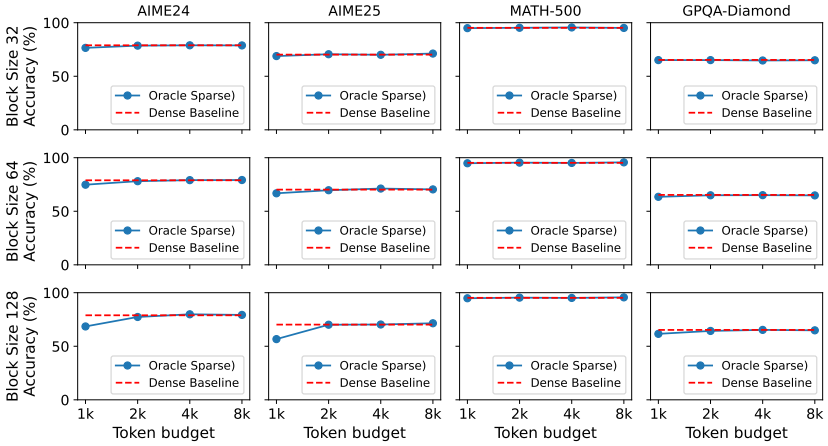


Figure 4: Oracle Sparse Results of Qwen3-14B with block size 32, 64, 128.

In the first experiment, we aim to answer the question: *How sparse is attention in reasoning models?* To investigate this, we employ *oracle block sparse selection*, which uses the same block-level attention score ground truth described in Section 2.3 (column-wise 1D max-pooling of the full attention map) to directly select the top-k KV blocks at each decoding step. This approach serves as an accuracy

upper bound: it reveals how well SeerAttention-R *could* perform if the gating mechanism perfectly predicted which blocks to attend. While this approach requires computing full attention to obtain the ground truth and does not provide any speedup, it allows us to evaluate the accuracy upper bound achievable by SeerAttention-R under ideal sparse selection.

We evaluate Qwen3-14B with three different sparse block sizes: 32, 64, and 128. The token budgets range from 1k to 8k. As shown in Figure 4, using oracle sparsity achieves lossless performance on all tasks when the token budgets reach 2k. For the more challenging AIME24 and AIME25 tasks, some accuracy degradation is observed with 1k token budget, particularly with the largest block size (128). However, this degradation is negligible when using block sizes of 32 or 64. These results indicate that attention sparsity exists in the reasoning process. Based on this, we select a block size of 64 as the default for SeerAttention-R.

4.3 Results of SeerAttention-R and Quest

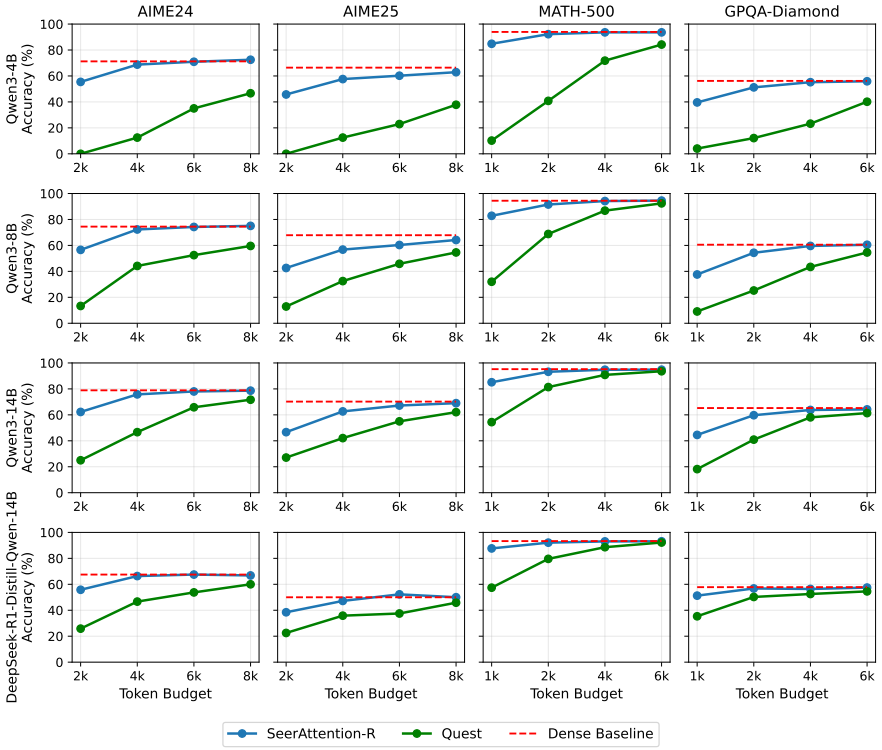


Figure 5: Accuracy Results of Full Attention, SeerAttention-R, and Quest. The Quest sparse configuration is set to be the same as SeerAttention-R for fair comparison, which uses a block size of 64 and sparse attention in all layers.

Figure 5 shows the results of all the models and benchmarks of Full Attention baseline, SeerAttention-R and Quest. As mentioned above, we modify the configuration of Quest to be the same as SeerAttention-R (block size 64 and using sparse attention in all layers). We use token budgets from 2k, 4k, 6k, and 8k for AIME24 and AIME25, and 1k, 2k, 4k, and 6k for MATH-500 and GPQA-Diamond. This is mainly because the typical averaged reasoning length from different benchmark is not the same. For the more challenging AIME24 and AIME25, the averaged generated lengths of these models are around 11k-18k. While for the easier MATH-500 and GPQA, the averaged lengths are reduced to 4k-9k. However, it is critical to note that across all combinations, the maximum generation lengths all reached the 32k token cap, underscoring the consistent demand for efficient long-context processing.

The results show that SeerAttention-R achieves consistently better performance compared to Quest. This trend holds true across every benchmark and computational budget, underscoring the robustness

and effectiveness of SeerAttention-R. For the AIME24 benchmark, SeerAttention-R typically achieves lossless performance with 4k token budget while the previous oracle sparse only requires 2k. This is within expectation as SeerAttention-R is only an approximation of ground truth with much less computation required. However, Quest fails to achieve lossless accuracy even using 8k token budgets under identical setting. For MATH-500 and GPQA-Diamond, the lossless token budgets reduce to 2k for SeerAttention-R while Quest requires around 8k to approach the full attention baseline.

A key trend observed across the results is the relationship between model scale and tolerance for sparse attention. Larger models, such as the 14B variants, exhibit greater robustness to the information loss inherent in sparsity compared to their 4B and 8B counterparts. This phenomenon is particularly pronounced for Quest, where the accuracy gap at lower budgets shrinks significantly as the model size increases. For SeerAttention-R, the effect is also present. The 14B models close the final gap to the dense baseline more easily than smaller models on challenging benchmarks like AIME25. This indicates that as reasoning models continue to scale, the viability of sparse attention methods increases.

In conclusion, the results demonstrate the superiority of SeerAttention-R's self-distilled approach over the training-free heuristics of Quest, especially in the challenging large block size configuration. Previous work LServe Yang et al. (2025b) also mentions the accuracy degradation of Quest over larger block sizes. They resolve this challenge by introducing Hierarchical Paging, a system approach that uses an additional level of block(page) abstraction called virtual logical page, which decouples the sparsity selection page size and physical page size. With SeerAttention-R, we can possibly simplify the sparse attention system design by using a larger block size.

4.4 Kernel Speedup

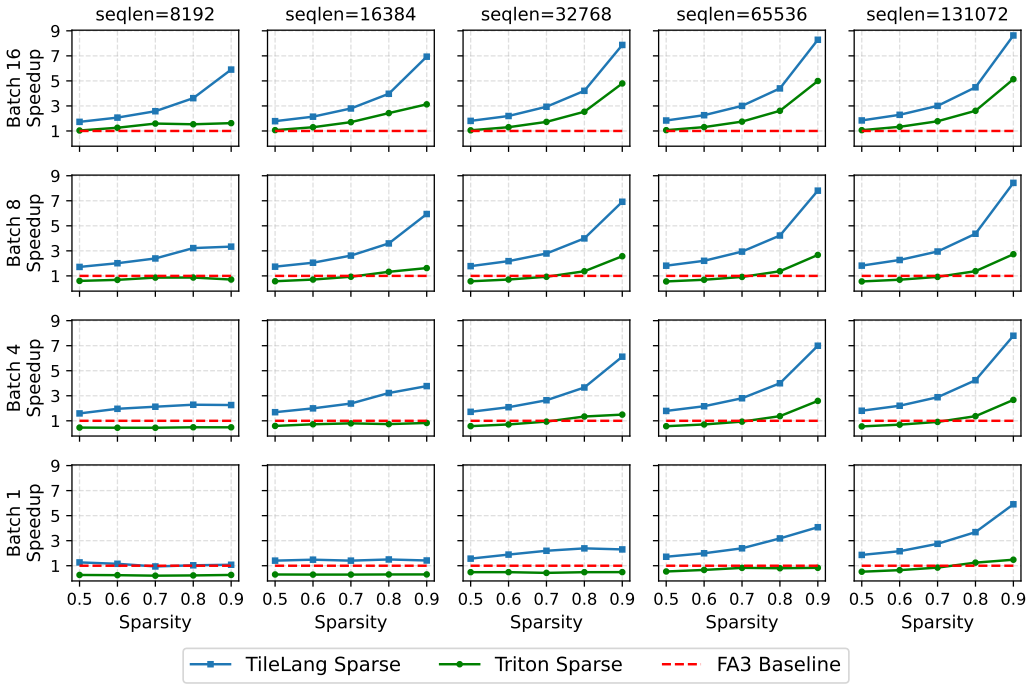


Figure 6: Kernel Speedup of our Block Sparse Flash-Decoding Kernel on H100 GPU. Our TileLang implementation of the kernel achieves higher speedup ratio compared to Triton implementation. For longer sequence length or larger batch size cases, the speedups approach the theoretical upper bound compared to FA3 basline.

This section evaluates our customized block sparse flash decoding kernel described in Section 3.3. We implement the kernel with both TileLang til and Triton and we use FlashAttention-3 (FA3) Shah et al. (2024) as baseline. The experiments are run on Nvidia H100 GPU with different input sequence

lengths (8k to 128k), batch sizes(1 to 16) and sparsity ratios (0.5 to 0.9). For the GQA configuration, we use 64 attention heads with 8 key-value heads, and head dimension 128.

Figure 6 presents the detailed results. Each subplot corresponds to a specific combination of input sequence length (seqlen) and batch size (bs), with the x-axis showing different sparsity ratios and the y-axis indicating speedup. The TileLang implementation consistently outperforms the FA3 baseline and achieves greater speedup than the Triton implementation. In general, the sparse kernel delivers higher speedup when the input sequence length is longer or the batch size is larger. This is expected, as the decoding kernel is primarily I/O-bound. When the KV cache size is sufficient to saturate the bandwidth, such as when bs=16 and seqlen $\geq$ 32k, our sparse kernel achieves near-theoretical speedup (up to $9\times$ at 0.9 sparsity). Even for moderate KV cache sizes, e.g. bs=4 and seqlen=32k, the kernel demonstrates significant speedup (up to $6\times$ at 0.9 sparsity).

5 Related Works

5.1 Training-free vs. Training-based Sparse Attention

Sparse attention research generally follows two directions: training-free (pre-defined or heuristic) and training-based methods. Training-free approaches adopt static patterns Xiao et al. (2023); Fu et al. (2024); Xiao et al. (2024b) or heuristic-based algorithms Zhang et al. (2023); Tang et al. (2024); Jiang et al. (2024); Yang et al. (2025b); Lai et al. (2025); Chen et al. (2024c); Liu et al. (2024c); Li et al. (2024a); Yang et al. (2024b); Hu et al. (2025); Zhang et al. (2025); Xu et al. (2025); Chen et al. (2025), relying on prior knowledge such as fixed patterns or head characteristics. In contrast, training-based methods integrate sparse attention into models to reduce complexity with minimal accuracy loss. Early work explored local/global/block patterns Child et al. (2019); Beltagy et al. (2020); Zaheer et al. (2020), while recent methods like NSA Yuan et al. (2025), MoBA Lu et al. (2025), ACP Lin et al. (2025b), MiniCPM4 Team (2025) train dynamic sparse modules during pre-training. SeerAttention offers a middle ground, learning sparsity post-training without modifying model weights.

5.2 KV Cache Compression

KV cache optimization is key for efficient inference, as smaller caches reduce bandwidth and memory costs. Eviction-based methods Ge et al. (2023); Li et al. (2024b); Zeng et al. (2024); Zhang et al. (2023); Liu et al. (2023); Adnan et al. (2024); Chen et al. (2024a); Behnam et al. (2025) permanently remove tokens, risking accuracy loss. Alternatively, dynamic selection methods Tang et al. (2024); Zhang et al. (2024); Hooper et al. (2024); Chen et al. (2025); Liu et al. (2024c); Hu et al. (2025); Cai et al. (2025); Hao et al. (2025); Mazaré et al. (2025a) retain all tokens but select subsets at each step.

5.3 Other Efficient Attention Algorithms

Beyond sparsity, efficient variants of multi-head attention include GQA Ainslie et al. (2023), MQA Shazeer (2019), latent-based designs Liu et al. (2024a); Zadouri et al. (2025), and cross-layer sharing approaches like YOCO Sun et al. (2024) and CLA Brandon et al. (2024). Linear attention Katharopoulos et al. (2020); Sun et al. (2023); Beck et al. (2024); Yang et al. (2024c); Gu and Dao (2023); Dao and Gu (2024); Peng et al. (2023); Yang et al. (2023) enables parallel training and constant inference memory, though it struggles in long-context reasoning. Hybrid models combining linear and full attention show stronger performance Dong et al. (2024); Li et al. (2025).

6 Conclusion and Discussion

This paper introduces SeerAttention-R, a lightweight sparse attention framework that accelerates long decoding in reasoning models. As a plug-in gating module, it integrates into pretrained models without altering original parameters and requires only lightweight training. Despite coarse-grained block sizes, SeerAttention-R preserves near-lossless reasoning accuracy, while its TileLang kernel achieves near-theoretical speedup at high sparsity ratios.

Several challenges remain. Achieving full end-to-end speedup will require integration with inference frameworks (e.g., vLLM Kwon et al. (2023), SGLang Zheng et al. (2024)) and compatibility with

PagedAttention, possibly combined with KV cache offloading Xiao et al. (2024a); Liu et al. (2024c); Chen et al. (2024c); Hao et al. (2025). Another open problem is adaptive sparsity, as the trade-off between accuracy and efficiency varies by task and sequence length. Top-p based sparsity selection Lin et al. (2025a); Chen et al. (2024c) offers one promising direction. Finally, unifying sparse prefill and decoding remains challenging: prefill benefits from parallelism while decoding does not. Approaches such as multi-token prediction Gloeckle et al. (2024); Liu et al. (2024b) or speculative decoding Leviathan et al. (2023) may help align them under a single gating mechanism.

In summary, SeerAttention-R shows that post-training sparse attention can deliver efficiency with minimal accuracy loss, while future work lies in adaptive sparsity, unified prefill/decoding, and system-level integration.

Limitations. The current evaluation of SeerAttention-R focuses on mathematical reasoning benchmarks (AIME24, AIME25, MATH-500, GPQA-Diamond) and does not cover other task domains such as long-context retrieval, code generation, or open-domain question answering. While we believe the learned sparsity patterns are general, transferability to non-mathematical tasks remains to be validated in future work. Additionally, our comparison is primarily against Quest; we did not compare with other post-training sparse attention methods for decode that are not evaluated on reasoning tasks such as TidalDecode (Yang et al., 2024a) and MagicPIG (Chen et al., 2024b). We also note a conceptual similarity to the later work DeepSeek Sparse Attention (DSA) (Liu et al., 2025a): both learn attention sparsity via self-distillation. The key difference is the attention variant: DSA targets MLA and achieves token-level selection.

ACKNOWLEDGEMENTS

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant No. U22B2022.

REFERENCES

TileLang. URL `https://github.com/tile-ai/tilelang`.

Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J Nair, Ilya Soloveychik, and Purushotham Kamath. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems*, 6:114–127, 2024.

Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.

Payman Behnam, Yaosheng Fu, Ritchie Zhao, Po-An Tsai, Zhiding Yu, and Alexey Tumanov. Rocketkv: Accelerating long-context llm inference via two-stage kv cache compression. *arXiv preprint arXiv:2502.14051*, 2025.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, Zhen Dong, Anima Anandkumar, Abedelkadir Asi, and Junjie Hu. R-kv: Redundancy-aware kv cache compression for training-free reasoning models acceleration. *arXiv preprint arXiv:2505.24133*, 2025.

Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, Weiyang Liu, and Chao Huang. Sepllm: Accelerate large language models by compressing one segment into one separator. *arXiv preprint arXiv:2412.12094*, 2024a.

Yaoqi Chen, Jinkai Zhang, Baotong Lu, Qianxi Zhang, Chengruidong Zhang, Jingjia Luo, Di Liu, Huiqiang Jiang, Qi Chen, Jing Liu, Bailu Ding, Xiao Yan, Jiawei Jiang, Chen Chen, Mingxing Zhang, Yuqing Yang, Fan Yang, and Mao Yang. Retroinfer: A vector-storage approach for scalable long-context llm inference, 2025. URL `https://arxiv.org/abs/2505.02922`.

Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, et al. Magicpig: Lsh sampling for efficient llm generation. *arXiv preprint arXiv:2410.16179*, 2024b.

Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, et al. Magicpig: Lsh sampling for efficient llm generation. *arXiv preprint arXiv:2410.16179*, 2024c.

Yu Cheng, Lei Wang, Yining Shi, Yuqing Xia, Lingxiao Ma, Jilong Xue, Yang Wang, Zhiwen Mo, Feiyang Chen, Fan Yang, Mao Yang, and Zhi Yang. PipeThreader: Software-defined pipelining for efficient dnn execution. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025. URL `https://www.usenix.org/conference/osdi25/presentation/cheng`.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. 2023. URL `https://arxiv.org/abs/2307.08691`.

Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, et al. Hymba: A hybrid-head architecture for small language models. *arXiv preprint arXiv:2411.13676*, 2024.

Hugging Face. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL `https://github.com/huggingface/open-r1`.

Tianyu Fu, Haofeng Huang, Xuefei Ning, Genghan Zhang, Boju Chen, Tianqi Wu, Hongyi Wang, Zixiao Huang, Shiyao Li, Shengen Yan, et al. Moa: Mixture of sparse attention for automatic large language model compression. *arXiv preprint arXiv:2406.14909*, 2024.

Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, et al. Seerattention: Learning intrinsic sparse attention in your llms. *arXiv preprint arXiv:2410.13276*, 2024.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Jitai Hao, Yuke Zhu, Tian Wang, Jun Yu, Xin Xin, Bo Zheng, Zhaochun Ren, and Sheng Guo. Omnikv: Dynamic context selection for efficient long-context llms. In *The Thirteenth International Conference on Learning Representations*, 2025.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Monishwaran Maheswaran, June Paik, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. Squeezed attention: Accelerating long context length llm inference. *arXiv preprint arXiv:2411.09688*, 2024.

Junhao Hu, Wenrui Huang, Weidong Wang, Zhenwen Li, Tiancheng Hu, Zhixia Liu, Xusheng Chen, Tao Xie, and Yizhou Shan. Efficient long-decoding inference with reasoning-aware attention sparsity. *arXiv preprint arXiv:2502.11147*, 2025.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *arXiv preprint arXiv:2407.02490*, 2024.

James M Joyce. Kullback-leibler divergence. In *International encyclopedia of statistical science*, pages 720–722. Springer, 2011.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. *arXiv preprint arXiv:2502.20766*, 2025.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025.

Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, et al. Scbench: A kv cache-centric analysis of long-context methods. *arXiv preprint arXiv:2412.10319*, 2024a.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024b.

Chaofan Lin, Jiaming Tang, Shuo Yang, Hanshuo Wang, Tian Tang, Boyu Tian, Ion Stoica, Song Han, and Mingyu Gao. Twilight: Adaptive attention sparsity with hierarchical top-p pruning. *arXiv preprint arXiv:2502.02770*, 2025a.

Zhixuan Lin, Johan Obando-Ceron, Xu Owen He, and Aaron Courville. Adaptive computation pruning for the forgetting transformer. *arXiv preprint arXiv:2504.06949*, 2025b.

Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024a.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024b.

Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025a.

Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, et al. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *arXiv preprint arXiv:2409.10516*, 2024c.

Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? an empirical study on quantized reasoning models. *arXiv preprint arXiv:2504.04823*, 2025b.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36:52342–52364, 2023.

Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Yan, Jianlin Su, Yuxin Wu, Yutao Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. Moba: Mixture of block attention for long-context llms. *arXiv preprint arXiv:2502.13189*, 2025.

Pierre-Emmanuel Mazaré, Gergely Szilvasy, Maria Lomeli, Francisco Massa, Naila Murray, Hervé Jégou, and Matthijs Douze. Inference-time sparse attention with asymmetric indexing. *arXiv preprint arXiv:2502.08246*, 2025a.

Pierre-Emmanuel Mazaré, Gergely Szilvasy, Maria Lomeli, Francisco Massa, Naila Murray, Hervé Jégou, and Matthijs Douze. Inference-time sparse attention with asymmetric indexing. *arXiv preprint arXiv:2502.08246*, 2025b.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37:68658–68685, 2024.

Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models. *Advances in Neural Information Processing Systems*, 37:7339–7361, 2024.

Yutao Sun, Tianzhu Ye, Dong Li, Yuqing Xia, Jian Chen, Yizhao Gao, Shijie Cao, Jianyong Wang, and Furu Wei. Rectified sparse attention. *arXiv preprint arXiv:2506.04108*, 2025.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.

MiniCPM Team. Minicpm4: Ultra-efficient llms on end devices. 2025.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

Lei Wang, Lingxiao Ma, Shijie Cao, Quanlu Zhang, Jilong Xue, Yining Shi, Ningxin Zheng, Ziming Miao, Fan Yang, Ting Cao, Yuqing Yang, and Mao Yang. Ladder: Enabling efficient low-precision deep learning computing through hardware-aware tensor transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 307–323, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-40-3. URL `https://www.usenix.org/conference/osdi24/presentation/wang-lei`.

Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. Infllm: Training-free long-context extrapolation for llms with an efficient context memory. *arXiv preprint arXiv:2402.04617*, 2024a.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024b.

Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. Xattention: Block sparse attention with antidiagonal scoring. *arXiv preprint arXiv:2503.16428*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.

Lijie Yang, Zhihao Zhang, Zhuofu Chen, Zikun Li, and Zhihao Jia. Tidaldecode: Fast and accurate llm decoding with position persistent sparse attention. *arXiv preprint arXiv:2410.05076*, 2024a.

Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. Lserve: Efficient long-sequence llm serving with unified sparse attention. *arXiv preprint arXiv:2502.14866*, 2025b.

Shuo Yang, Ying Sheng, Joseph E Gonzalez, Ion Stoica, and Lianmin Zheng. Post-training sparse attention with double sparsity. *arXiv preprint arXiv:2408.07092*, 2024b.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024c.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.

Ted Zadouri, Hubert Strauss, and Tri Dao. Hardware-efficient attention for fast decoding. *arXiv preprint arXiv:2505.21487*, 2025.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.

Zihao Zeng, Bokai Lin, Tianqi Hou, Hao Zhang, and Zhijie Deng. In-context kv-cache eviction for llms via attention-gate. *arXiv preprint arXiv:2410.12876*, 2024.

Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kvcache for long context llm inference. *arXiv preprint arXiv:2407.12820*, 2024.

Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. Spargeattn: Accurate sparse attention accelerating any model inference. In *International Conference on Machine Learning (ICML)*, 2025.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=RkRrPp7GKO.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37: 62557–62583, 2024.

Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, Fan Yang, Mao Yang, Lidong Zhou, Asaf Cidon, and Gennady Pekhimenko. ROLLER: Fast and efficient tensor compilation for deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 233–248, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-28-1. URL https://www.usenix.org/conference/osdi22/presentation/zhu.

A APPENDIX

A.1 BLOCK SIZE FOR SPARSE ATTENTION

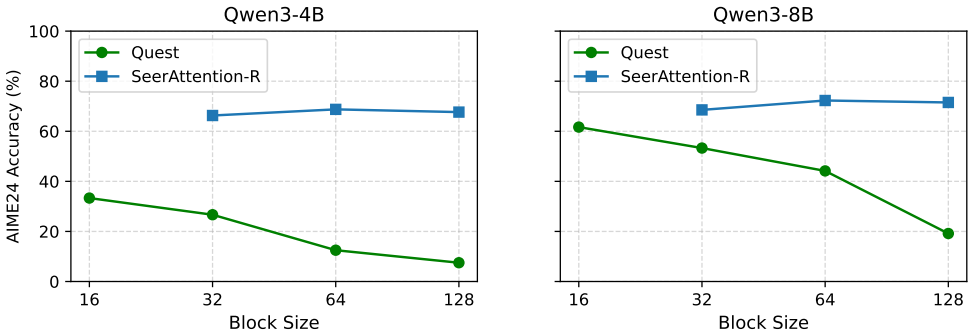


Figure 7: AIME24 results using different block sizes with 4k token budget. SeerAttention-R achieves almost consistent performances on different block sizes. However, Quest gets lower accuracy when block size gets larger. Note that in this experiment, SeerAttention-R enables shared sparsity selection within each GQA group, whereas Quest does not.

The token block size for sparse attention is a critical factor that affects overall system performance. If the block size is too small, it incurs significant overhead in sparse block prediction, including increased computational cost and larger metadata requirements such as compression caches and block indices. While a larger block size can also potentially improve the utilization of GPUs.

Figure 7 presents AIME24 results on the Qwen3-4B and Qwen3-8B models across block sizes ranging from 16 to 128. By default, Quest uses a block size of 16. The results indicate that Quest's performance decreases as the block size increases. However, SeerAttention-R achieves consistent accurate sparse block selection at different block sizes. Remarkably, this robustness lies under the assumption of the additional mask sharing in the GQA group dimension. We excluded a block size of 16 from our experiments due to its inefficiency during both training and inference. It often leads to out-of-memory errors because of the large intermediate attention maps generated during training.

A.2 HYBRID DENSE ATTENTION IN THE FIRST TWO LAYERS

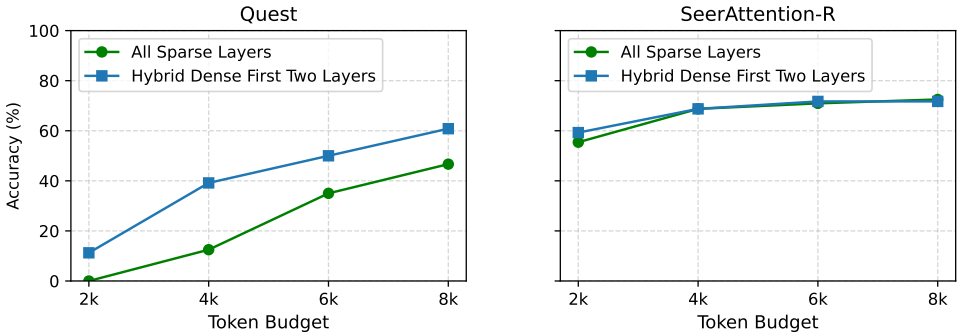
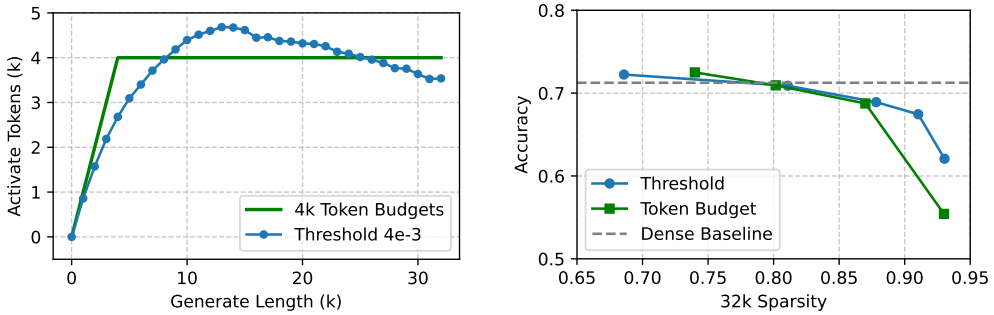


Figure 8: AIME24 results of whether using dense attention in first two layers (Qwen3-4B).

Some post-training sparse attention algorithms employ hybrid dense attention in certain layers to mitigate accuracy loss. By default, Quest applies dense attention in its first two layers. However, for a fair comparison, we evaluate both Quest and SeerAttention-R using purely sparse attention across all layers in previous evaluation. This approach allows us to isolate and analyze the effects of sparse attention without the confounding influence of hybrid attention.

To further investigate the impact of hybrid dense attention, we conduct an ablation study using the Qwen3-4B model on the AIME24 benchmark with a block size of 64. As shown in Figure 8, incorporating hybrid dense attention in Quest yields a significant improvement in accuracy, whereas SeerAttention-R only sees marginal benefits. This difference may be due to the already accurate sparse prediction by SeerAttention-R in the first two layers, reducing the potential gains from hybridization.

A.3 THRESHOLD VS TOKEN BUDGETS



(a) (b)

Figure 9: Threshold vs. Token Budget. Results are obtained using Qwen3-4B models on AIME24 benchmark. (a) Difference of activated tokens distribution of two methods. (b) Sparsity vs Accuracy tradeoff of two methods. Thresholds: 2e-3, 3e-3, 4e-3, 5e-3, 6e-3. Token Budget: 8k, 6k, 4k, 2k.

In SeerAttention-R, we employ two AttnGate sparsification strategies, threshold and token budget, to convert real-valued gate scores into discrete block selections. The token budget method offers an straightforward way to align sparsity and compare with different methods. However, the threshold method is extremely simple to implement and avoids the need of sorting. Figure 9a illustrates the distribution of activated tokens across varying sequence lengths using a threshold of 4e-3 and a token budget of 4K on the AIME24 benchmark with Qwen3-4B model. The token budget approach results in a strict piecewise linear activation pattern, whereas the threshold method yields a smoother, curved distribution. Figure 9b compares the sparsity–accuracy trade-offs of the two methods. The threshold method shows slightly better accuracy in high sparsity region.

A.4 IMPACT OF SPARSE ATTENTION ON GENERATE LENGTH

Table 1: Qwen3-8B AIME24 Accuracy vs. Reasoning Length.

		Token Budgets			
		2k	4k	6k	8k
Quest	Accuracy	13.3	44.2	52.5	59.6
	Gen. Length(k)	30.0	22.9	19.6	17.2
SeerAttention-R	Accuracy	56.6	72.3	74.2	75.1
	Gen. Length(k)	19.8	16.3	15.3	15.1

We observed that using inaccurate sparse attention (too small budget or low recall) can increase output token lengths in reasoning tasks. Table 1 shows the AIME accuracy and reasoning length using Qwen3-8B model. The baseline accuracy of full attention and the generated length are 74.5 and 15.1 k, respectively. We can see that Quest, and SeerAttention-R with 2k budget cases, all incur much longer reasoning paths compared to full attention. A similar phenomenon has been reported in quantization Liu et al. (2025b), where inaccurate quantization algorithms lead to longer reasoning paths. We believe this effect is universal across different post-training efficiency optimizations of reasoning model, as such methods can introduce errors that accumulate over the long reasoning chains. These additional reasoning steps potentially undermine the original goal of improving efficiency. Therefore, an accurate sparse attention selection algorithm is crucial to mitigate this effect. Another promising approach to eliminate the accumulated errors is to use Rectified Sparse Attention Sun et al. (2025), which periodically performs dense rectification of the KV cache.

A.5 TRAINING BUDGET

Table 2: Training Budgets

Training Tokens	**GPU Hours**		
0.4B	Qwen3-4B 10.9	Qwen3-8B 12.2	Qwen3-14B 18.6

As a lightweight distillation process where only the AttnGate parameters are trained, SeerAttention-R is also highly efficient in terms of training. In our experiments, we set the global batch size to 16 and trained for just 800 steps, utilizing DeepSpeed Stage 2 optimization on MI300x GPUs. Each data batch is packed to a sequence length of 32k with our custom variable-length FlashAttention forward kernel, as described in Section 2.3. Table 2 summarizes the GPU hours required for training models of various sizes. Notably, distilling an 8B model requires only 12 GPU hours, demonstrating the efficiency of our approach. Increasing the quantity, quality, and diversity of training data may lead to further improvements.